

A REPORT ON THE PROJECT ENTITLED
**DYNAMIC BINARY INSTRUMENTATION USING
INTEL PIN TOOL**

VI SEMESTER B-TECH CSE
**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SURATHKAL**

2025 – 2026

GROUP MEMBERS:

**ADITHYA B M
221CS104**

**ARUN M MYAGERI
221CS113**

**AMIT KUMAR
221CS207**

**RAHUL MONDAL
221CS142**

**SAGNIK DAS
221CS147**

**VIVEK KUMAR
221CS166**

Tasks to do / Questions to answer.

1. Find out what is Dynamic Binary Instrumentation. Where and what for is this used?

Dynamic Binary Instrumentation (DBI) is a technique used to analyze or modify the behavior of a binary program at runtime without requiring access to its source code. DBI works by injecting instrumentation code into a running application, allowing developers or researchers to monitor, analyze, or change its execution dynamically.

Key Features of DBI:

1. **Dynamic Nature:** It works at runtime, unlike static analysis, which examines code before execution.
2. **Binary-Level:** Operates directly on compiled binaries, not source code.
3. **Flexible Instrumentation:** Users can insert custom logic to monitor or alter the behavior of specific instructions or code blocks.
4. **Transparent:** The original program's functionality is preserved (unless explicitly modified).

Where Is DBI Used?

DBI is applied across several domains, including:

1. Software Performance Profiling:

- ☐ DBI can analyze performance bottlenecks, memory usage, and hotspots in an application.
- ☐ Examples: Tools like *Intel Pin* or *DynamoRIO* are used to collect performance metrics.

2. Debugging and Testing:

- ☐ Helps in tracking bugs or unusual behaviors by monitoring program execution.
- ☐ It can also be used for coverage analysis in testing.

3. Security:

- ☐ Analyzing binary programs for vulnerabilities, malware behavior, or potential exploits.
- ☐ Dynamic analysis tools can sandbox suspicious binaries to observe behavior in real time.

4. Instruction-Level Monitoring:

- ☐ Enables the study of individual instructions to understand program behavior or optimize code.

5. Runtime Modification:

- ☐ DBI can modify execution at runtime, enabling optimizations, sandboxing, or emulation.

Examples of DBI Frameworks:

1. Pin (Intel):

- ☐ Widely used for building custom program analysis tools.
- ☐ Provides APIs to instrument code dynamically.

2. DynamoRIO:

- ☐ Open-source DBI framework used for analysis, optimization, and security applications.

3. Valgrind:

- ☐ A framework that uses DBI to create tools for memory debugging, leak detection, and profiling.

4. Frida:

- ☐ Popular for security-related tasks, such as real-time application instrumentation.

Why Use DBI?

- ☐ **No Source Code Required:** Ideal for proprietary or legacy binaries where the source code is unavailable.
- ☐ **Real-Time Monitoring:** Analyze how the binary interacts with the system and track runtime behaviors.
- ☐ **Flexible:** Provides granular control to instrument as little or as much as needed.
- ☐ **Non-Intrusive:** Binaries do not need recompilation, reducing overhead.

2. . Install and run all the examples.

Step 1:

go through the website and download latest version of pin

<https://www.intel.com/content/www/us/en/developer/articles/tool/pin-a-binary-instrumentation-tool-downloads.html>

> Downloads

[Related Content](#)

[Related Products](#)

Tool Overview

- Pin is a dynamic binary instrumentation framework for the IA-32, x86-64 and MIC instruction-set architectures.
- Enables the creation of dynamic program analysis tools.
- Has a diverse set of tools for security, emulation and parallel program analysis.

[Learn More](#)

Downloads

Download Pin kits here. Before using Pin, please read the [license](#). For questions or comments, please go to pinheads@groups.io.

Linux*

IA32 and intel64 (x86 32 bit and 64 bit)

Version	Date	Kit	Signature	Documentation		
Pin 3.31	June 30, 2024	98869	sig	Manual	PinCRT	Release Notes
Pin 3.30	January 9, 2024	98830	sig	Manual	PinCRT	Release Notes
Pin 3.29	September 30, 2023	98790	sig	Manual	PinCRT	Release Notes
Pin 3.28	July 3, 2023	98749	sig	Manual	PinCRT	Release Notes
Pin 3.27	April 1, 2023	98718	sig	Manual	PinCRT	Release Notes
Pin 3.26	January 15, 2023	98690	sig	Manual	PinCRT	Release Notes
Pin 3.25	October 20, 2022	98650	sig	Manual	PinCRT	Release Notes
Pin 3.24	July 18, 2022	98612	sig	Manual	PinCRT	Release Notes
Pin 3.23	May 12, 2022	98579	sig	Manual	PinCRT	Release Notes
Pin 3.22	February 28, 2022	98547	sig	Manual	PinCRT	Release Notes
Pin 3.21	October 28, 2021	98484	sig	Manual	PinCRT	Release Notes
Pin 3.20	July 12, 2021	98437	sig	Manual	PinCRT	Release Notes

step 2:

Extracting all the files from file named as “pin-external-3.31-98869-gfa6f126a8-gcc-linux.tar.gz”
3.31 is a Version of PinTool.

```
roy@roy-Inspiron-14-5430: ~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples
roy@roy-Inspiron-14-5430:~$ cd Do
Documents/ Downloads/
roy@roy-Inspiron-14-5430:~/Downloads$ cd Downloads/
roy@roy-Inspiron-14-5430:~/Downloads$ ls
pin-external-3.31-98869-gfa6f126a8-gcc-linux.tar.gz
'pin-external-3.31-98869-gfa6f126a8-gcc-linux.tar.gz(1).sig'
pin-external-3.31-98869-gfa6f126a8-gcc-linux.tar.gz.sig
roy@roy-Inspiron-14-5430:~/Downloads$ rm -rf ~/Downloads/*
roy@roy-Inspiron-14-5430:~/Downloads$ ls
roy@roy-Inspiron-14-5430:~/Downloads$ tar -xvzf pin-external-3.31-98869-gfa6f126a8-gcc-linux.tar.gz
pin-external-3.31-98869-gfa6f126a8-gcc-linux/
pin-external-3.31-98869-gfa6f126a8-gcc-linux/README
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/debugger-protocol.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/ops.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/ops-enum.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/exponential-backoff.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/private/
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/private/intel64/
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/private/intel64/ops-impl.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/private/intel64/ops-impl-intel64-asm.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/private/backoff-impl.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/private/ops-common-impl.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/private/ia32/
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/private/ia32/ops-impl.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/private/ia32/ops-impl-ia32-asm.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/fixd-lifo.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/idset.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/nullstats.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/config.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/lifo-ptr.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/fixd-multiset.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/lifo-ctr.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/atomic/fixd-multimap.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/asm/
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/asm/gas-x86.h
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/asm/masm-x86.h
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/asm/windows-intel64.h
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/asm/masm-x86.h
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/asm/unix-intel64.h
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/types.h
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/util/
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/util/regvalue.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/util/range.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/util/singleton.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/util/data.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/util/quote-argument-ms.hpp
pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/components/include/util/numberbits.hpp
```

Step 3:

- ☐ Go to source/tools/ManualExamples
 - `cd /opt/pin-dir/source/tools`
- ☐ In this directory all the files required for counting instruction (inscount0.cpp) , pint tracing(pintrace.cpp) are exist.
- ☐ Check whether you have a 32 bit architecture or a 64 bit architecture using `lscpu` command. Depending on this your compiler arguments will have to be modified. If the architecture is `x86_64`.
- ☐ make all
make all TARGET=intel64 (if architecture is 64bit).

We found that latest version works better with intel64 than ia32 due to compatibility issues


```

roy@roy-inspiron-14-5430: ~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples
roy@roy-inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ make inscount0.test TARGET=intel64
mkdir -p obj-intel64/
g++ -Wall -Werror -Wno-unknown-pragmas -DPIN_CRT=1 -fno-stack-protector -fno-exceptions -funwind-tables -fasynchronous-unwind-tables -fno-rtti -DTARGET_IA32E -DHOST_IA32E -fPIC -DTARGET_LINUX
-X -fabi-version=2 -faligned-new -I../..../source/include/pin -I../..../source/include/pin/gen -isystem /home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/cxx/include -
isystem /home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include -isystem /home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include/arch-
x86_64 -isystem /home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include/kernel/uapi -isystem /home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extr
as/crt/include/kernel/uapi/asm-x86 -I../..../extras/components/include -I../..../extras/xed-intel64/include/xed -I../..../source/tools/Utils -I../..../source/tools/InstLib -O3 -fomit-fra
me-pointer -fno-strict-aliasing -Wno-dangling-pointer -c -o obj-intel64/inscount0.o inscount0.cpp
g++ -shared -Wl,--hash-style=sysv ../..../intel64/runtime/pincrt/crtbegin.o -Wl,-Bsymbolic -Wl,--version-script=../..../source/include/pin/pintool.ver -fabi-version=2 -o obj-intel64/ins
count0.so obj-intel64/inscount0.o -L../..../intel64/runtime/pincrt -L../..../intel64/lib -L../..../intel64/lib-ext -L../..../extras/xed-intel64/lib -lpin -lxed ../..../intel64/runtime/p
incrt/crtend.o -lpdwarf -ldwarf -ldl -dynamic -nostdlib -lc++ -lc++abi -ln-dynamic -lc-dynamic -lunwind-dynamic
/usr/bin/ld: warning: uttl_host_ia32e.spp.o: missing .note.GNU-stack section implies executable stack
/usr/bin/ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
make -C ../..../source/tools/Utils dir obj-intel64/cp-pin.exe
make[1]: Entering directory '/home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/Utils'
mkdir -p obj-intel64/
g++ -DTARGET_IA32E -DHOST_IA32E -DFUND_TC_TARGETCPU=FUND_CPU_INTEL64 -DFUND_TC_HOSTCPU=FUND_CPU_INTEL64 -DTARGET_LINUX -DFUND_TC_TARGETOS=FUND_OS_LINUX -DFUND_TC_HOSTOS=FUND_OS_LINUX -I../..
../source/tools/Utils -O3 -std=c++11 -o obj-intel64/cp-pin.exe cp-pin.cpp -no-pie
make[1]: Leaving directory '/home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/Utils'
../..../pin -t obj-intel64/inscount0.so -- ../..../source/tools/Utils/obj-intel64/cp-pin.exe makefile obj-intel64/inscount0.makefile.copy \
> obj-intel64/inscount0.out 2>&1
cmp makefile obj-intel64/inscount0.makefile.copy
rm obj-intel64/inscount0.makefile.copy
rm obj-intel64/inscount0.out
roy@roy-inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ ../..../pin -t obj-intel64/inscount0.so -- /bin/ls
buffer_linux.cpp      dumpargv.cpp      fork_app.cpp      inscount.out      makefile          obj-intel64      stack-debugger-tutorial.sln      w_malloctrace.cpp
buffer_windows.cpp    emuldiv.cpp       fork_jit_tool.cpp inscount_tls.cpp  makefile.rules   pinatrace.cpp   stack-debugger-tutorial.vcxproj
countreps.cpp         fibonacci.cpp      imageload.cpp     invocation.cpp    malloc_mt.cpp    proccount.cpp   stack-debugger-tutorial.vcxproj.filters
detach.cpp            follow_child_app1.cpp inscount0.cpp     isamplng.cpp     malloctrace.cpp  replacesigprobed.cpp statica.cpp
divide_by_zero_unix.c follow_child_app2.cpp inscount1.cpp     itrace.cpp       nonstatica.cpp  safecopy.cpp    staticcount.cpp  staticcount.cpp
divide_by_zero_win.c  follow_child_tool.cpp inscount2.cpp     little_malloc.c  obj-ia32        stack-debugger.cpp strace.cpp
roy@roy-inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ cat inscount.out
Count 671501
roy@roy-inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ nano print_n_integers.cpp
roy@roy-inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ g++ -o print_n_integers print_n_integers.cpp
roy@roy-inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ ../..../pin -t obj-intel64/inscount0.so -- ./print_n_integers
Enter the number of integers to print: 10
1
2
3
4
5
6
7
8
9
10
roy@roy-inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ cat inscount.out
Count 1791342
roy@roy-inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ nano count_instructions.cpp
roy@roy-inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$

```

For running any particular program file (.cpp) commands are:

1. nano file_name.cpp
2. gcc -o file_name file_name.cpp (it will create a.out file)
3. ../..../pin -t obj-intel64/inscount0.so -- ./file_name (it creates inscount.out)
4. cat inscount.out (final output)

3. Learn how to use it for counting instructions, counting specific type of instructions (Eg. loads/ stores / ALU /).

For counting loads/stores/ALU :

1. We created a file name called count_instructions.cpp with modification of inscount0.cpp

Code:

```
#include "pin.H"
```

```
// Global variables to count loads, stores, arithmetic, and logic instructions
```

```
static UINT64 loadCount = 0;
```

```
static UINT64 storeCount = 0;
```

```
static UINT64 arithmeticCount = 0;
```

```
static UINT64 logicCount = 0;
```

```

// Function to count instructions
VOID Instruction(INS ins, VOID* v)
{
    // Check if the instruction is a load
    if (INS_IsMemoryRead(ins)) {
        loadCount++;
    }

    // Check if the instruction is a store
    if (INS_IsMemoryWrite(ins)) {
        storeCount++;
    }

    // Check for arithmetic instructions manually
    if (INS_Opcode(ins) == XED_ICLASS_ADD || INS_Opcode(ins) ==
XED_ICLASS_SUB||
INS_Opcode(ins) == XED_ICLASS_MUL || INS_Opcode(ins) == XED_ICLASS_DIV)
    {
        arithmeticCount++;
    }

    // Check for logic instructions manually
    if (INS_Opcode(ins) == XED_ICLASS_AND || INS_Opcode(ins) ==
XED_ICLASS_OR ||
INS_Opcode(ins) == XED_ICLASS_XOR || INS_Opcode(ins) == XED_ICLASS_SHL
||
INS_Opcode(ins) == XED_ICLASS_SHR)

        logicCount++;
    }
}

// Function to print the results
VOID Fini(INT32 code, VOID* v)
{
    printf("Loads: %lu\n", loadCount);
    printf("Stores: %lu\n", storeCount);
    printf("Arithmetic: %lu\n", arithmeticCount);
    printf("Logic: %lu\n", logicCount);
}

// Main function to set up the analysis
int main(int argc, char* argv[])
{
    // Initialize Pin
    PIN_Init(argc, argv);

    // Register the instruction callback
    INS_AddInstrumentFunction(Instruction, 0);

    // Register the finalization function
    PIN_AddFiniFunction(Fini, 0);
}

```



```
// Start the program
PIN_StartProgram();
return 0; }
```

OUTPUT :

Input file is : alu_count.cpp

```
#include <iostream>
int main() {
    int a, b;
    // Input two integers
    std::cout << "Enter two integers: ";
    std::cin >> a >> b;
    // Perform basic operations
    int sum = a + b;
    int diff = a - b;
    int product = a * b;
    int quotient = (b != 0) ? a / b : 0; // Avoid division by zero
    // Display results
    std::cout << "Sum: " << sum << std::endl;
    std::cout << "Difference: " << diff << std::endl;
    std::cout << "Product: " << product << std::endl;
    std::cout << "Quotient: " << quotient << std::endl;
    return 0;
}
```

```
roy@roy-Inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ nano count_instructions.cpp
roy@roy-Inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ make count_instructions.test TARGET=intel64
g++ -Wall -Werror -Wno-unknown-pragmas -DPIN_CRT=1 -fno-stack-protector -fno-exceptions -funwind-tables -fasynchronous-unwind-tables -fno-rtti -DTARGET_IA32E -DHOST_IA32E -FPIC -DTARGET_LINU
X -fabi-version=2 -faligned-new -I../..../source/include/pin -I../..../source/include/pin/gen -isystem /home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/cxx/include -
isystem /home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include -isystem /home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include/arch-
x86_64 -isystem /home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include/kernel/uapi -isystem /home/roy/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extr
as/crt/include/kernel/uapi/asm-x86 -I../..../extras/components/include -I../..../extras/xed-intel64/include/xed -I../..../source/tools/Utils -I../..../source/tools/InstLib -O3 -fomit-fra
me-pointer -fno-strict-aliasing -Wno-dangling-pointer -c -o obj-intel64/count_instructions.o count_instructions.cpp
g++ -shared -Wl,--hash-style=sysv ../..../intel64/runtime/pincrt/crtbegin5.o -Wl,-Bsymbolic -Wl,--version-script=../..../source/include/pin/pintool.ver -fabi-version=2 -o obj-intel64/cou
nt_instructions.so obj-intel64/count_instructions.o -L../..../intel64/runtime/pincrt -L../..../intel64/lib -L../..../intel64/lib-ext -L../..../extras/xed-intel64/lib -lpin -lxed ../..../
intel64/runtime/pincrt/crtend5.o -lpindwarf -ldwarf -ldl -dynamic -nostdlib -lc++ -lc++abi -ln-dynamic -lc-dynamic -lunwind-dynamic
/usr/bin/ld: warning: util_host_ia32e.spp.o: missing .note.GNU-stack section implies executable stack
/usr/bin/ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
../..../pin -t obj-intel64/count_instructions.so -- ../..../source/tools/Utils/obj-intel64/cp-pin.exe makefile obj-intel64/count_instructions.makefile.copy \
> obj-intel64/count_instructions.out 2>&1
cmp makefile obj-intel64/count_instructions.makefile.copy
rm obj-intel64/count_instructions.makefile.copy
rm obj-intel64/count_instructions.out
roy@roy-Inspiron-14-5430:~/Downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples$ ../..../pin -t obj-intel64/count_instructions.so -- ./alu_count
Enter two integers: 2 3
Sum: 5
Difference: -1
Product: 6
Quotient: 0
Loads: 7642
Stores: 5202
Arithmetic: 1895
Logic: 1739
```